

AI SCIENTIST

FASE 2 | AULA 02 -

MODELAGEM DE DADOS EM NOSQL: CHAVE-VALOR, DOCUMENTOS, COLUNAS E GRAFOS

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 5
MERCADO, CASES E TENDÊNCIAS 17
O QUE VOCÊ VIU NESTA AULA? 20
REFERÊNCIAS 21

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

O QUE VEM POR AÍ?

Nos últimos anos, o volume, a variedade e a velocidade dos dados cresceram em um ritmo que desafia diretamente os modelos tradicionais de banco de dados. Aplicações modernas, como plataformas digitais, redes sociais, sistemas de recomendação e soluções em tempo real, passaram a exigir estruturas mais flexíveis, escaláveis e adaptadas a diferentes tipos de informação. É nesse cenário que a modelagem de dados em **NoSQL** ganha protagonismo, não como substituta absoluta do modelo relacional, mas como uma alternativa estratégica para problemas específicos.

Nesta aula, você irá repensar a forma de modelar dados a partir de quatro grandes abordagens do universo NoSQL: **chave-valor, documentos, colunas e grafos**. Vamos explorar como cada modelo organiza a informação, quais problemas resolve melhor e como as decisões de modelagem impactam diretamente desempenho, escalabilidade e facilidade de evolução das aplicações.

Ao longo do conteúdo, reflita: o dado deve se adaptar ao banco ou o banco ao dado? Essa pergunta será o ponto de partida para tudo o que vem a seguir.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

HANDS ON

Modelagem de Dados em NoSQL no Databricks

Nesta etapa prática, vamos colocar a mão na massa e entender **como a modelagem de dados em NoSQL se materializa na prática**, utilizando o **Databricks** como ambiente de execução. O objetivo do hands on é demonstrar, de forma guiada e progressiva, como diferentes modelos NoSQL podem ser representados e explorados dentro de uma plataforma analítica moderna, aproveitando o Apache Spark e estruturas flexíveis como JSON, mapas, arrays e grafos.

Ao longo deste exercício, você irá criar exemplos simples, porém realistas, que simulam cenários comuns de aplicações digitais. A ideia não é apenas executar código, mas **entender as decisões de modelagem** por trás de cada estrutura, observando como os dados são organizados, acessados e analisados. Recomenda-se acompanhar o passo a passo diretamente em um notebook Databricks, executando cada célula e analisando os resultados antes de avançar.

Veja o passo a passo na seção “Saiba mais”.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

SAIBA MAIS

Passo a passo — Preparando o Ambiente

Passo 1 — Acessar o Databricks Free

Acesse: https://community.cloud.databricks.com, faça login ou crie uma conta gratuita. Após o login, você será direcionado ao **Workspace.**

Passo 2 — Criar um Notebook

No menu lateral esquerdo, clique em **Workspace.** Clique com o botão direito → **Create** → **Notebook**. Nomeie o notebook, por exemplo: HandsOn_NoSQL_Databricks

Selecione:

Language: Python

Cluster: Starter (padrão do Community Edition)

Passo 3 — Validar o Ambiente

Execute a célula abaixo para validar o ambiente Spark: python

```
spark
```

Se o objeto SparkSession for exibido, o ambiente está pronto.

Passo 4 – Modelo Chave-Valor

O modelo **chave-valor** é o mais simples do NoSQL. Cada registro é identificado por uma chave única, associada a um valor, que pode ser simples ou complexo.

```
from pyspark.sql import Row


dados_kv = [
    Row(chave="user:1", valor={"nome": "Ana", "idade": 29}),
    Row(chave="user:2", valor={"nome": "Bruno", "idade": 35}),
    Row(chave="user:3", valor={"nome": "Carla", "idade": 22})
```

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

```
]


df_kv = spark.createDataFrame(dados_kv)
df_kv.show(truncate=False)
```

Código-fonte 1 – Exemplo de código fonte python

Fonte: Elaborado pelo autor (2026)

Aqui, observe que **não há esquema rígido** para o valor. Essa abordagem é muito comum em caches distribuídos e sessões de usuários. No Databricks, conseguimos analisar esse tipo de estrutura sem precisar transformá-la previamente em tabelas relacionais.

Passo 5 – Modelo orientado a documentos

No modelo de **documentos**, cada registro é um documento autônomo, geralmente em formato JSON, que agrupa dados relacionados.

```
dados_documentos = [
  {
    "id": 1,
    "nome": "Ana",
    "enderecos": [
      {"cidade": "São Paulo", "tipo": "Residencial"},
      {"cidade": "Campinas", "tipo": "Trabalho"}
    ]
  },
  {
    "id": 2,
    "nome": "Bruno",
    "enderecos": [
```

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

```
            {"cidade": "Rio de Janeiro", "tipo": "Residencial"}
        ]
    }
]

df_docs = spark.read.json(
    spark.sparkContext.parallelize(dados_documentos)
)

df_docs.printSchema()
df_docs.show(truncate=False)
```

Código-fonte 2 – Exemplo de código fonte python

Fonte: Elaborado pelo autor (2026)

Repare no **uso de arrays e structs**, que permitem modelar relações complexas sem a necessidade de joins. Esse tipo de modelagem é muito usado em bancos como MongoDB e se encaixa perfeitamente em pipelines analíticos no Databricks.

Passo 6 – Modelo orientado a colunas

No modelo de **colunas**, os dados são organizados por famílias de colunas, otimizando leituras analíticas e consultas em grande escala.

```
dados_colunas = [
    ("user1", "2024-01", 120.5, 3),
    ("user1", "2024-02", 98.0, 2),
    ("user2", "2024-01", 210.0, 5)
]

df_colunas = spark.createDataFrame(
```

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

```
    dados_colunas,
    ["usuario", "mes", "valor_compras", "qtd_compras"]
)

df_colunas.show()
```

Código-fonte 3 – Exemplo de código fonte python

Fonte: Elaborado pelo autor (2026)

Esse tipo de estrutura se aproxima bastante de soluções como Cassandra ou HBase e conversa diretamente com o conceito de **dados distribuídos e analíticos**, algo muito explorado no ecossistema Databricks.

Passo 7 – Modelo Orientado a Grafos

No modelo de **grafos**, o foco está nas relações. Vamos simular um pequeno grafo de conexões entre usuários.

```
vertices = spark.createDataFrame([
    (1, "Ana"),
    (2, "Bruno"),
    (3, "Carla")
], ["id", "nome"])

arestas = spark.createDataFrame([
    (1, 2, "segue"),
    (2, 3, "segue"),
    (1, 3, "amigo")
], ["origem", "destino", "relacao"])

vertices.show()
```

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

```
arestas.show()
```

Código-fonte 4 – Exemplo de código fonte python
Fonte: Elaborado pelo autor (2026)

Mesmo sem uma engine específica de grafos, já é possível analisar relações e construir consultas analíticas. Em ambientes avançados, essa modelagem se integra a bibliotecas de grafos para análises de recomendação, influência e redes complexas.

Ao concluir este hands on, volte aos notebooks e reflita sobre como **a modelagem de dados em NoSQL é orientada ao acesso e ao problema**, e não à normalização clássica. No Databricks, esses modelos coexistem de forma natural, permitindo que diferentes estratégias de armazenamento e análise sejam usadas dentro da mesma arquitetura.

Antes de avançar, revise cada exemplo, altere os dados e observe como pequenas mudanças de modelagem impactam diretamente a forma de consultar e analisar a informação.

Modelagem de Dados em NoSQL: conceitos, estratégias e exemplos

Vamos aprofundar os conceitos apresentados no hands on e **expandir a compreensão sobre modelagem de dados em NoSQL**, explorando os quatro principais modelos: **chave-valor, documentos, colunas e grafos**. O foco aqui não é apenas entender “como funciona”, mas **por que modelar de determinada forma**, quais decisões arquiteturais estão envolvidas e quais impactos essas escolhas trazem para escalabilidade, desempenho, manutenção e evolução das aplicações.

Ao longo do texto, conecte os conceitos apresentados com os exemplos práticos vistos no Databricks e reflita sobre como essas abordagens se encaixam em arquiteturas modernas orientadas a microsserviços, eventos e analytics em larga escala.

A lógica da modelagem em NoSQL

Diferentemente do modelo relacional, cuja modelagem é guiada por normalização, integridade referencial e redução de redundância, a modelagem em

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

NoSQL é **orientada ao padrão de acesso aos dados**. Em outras palavras, começa-se pela pergunta: como os dados serão lidos e escritos pela aplicação?

Essa mudança de mentalidade é fundamental. Em NoSQL, é comum aceitar redundância de dados, estruturas aninhadas e até inconsistência temporária, desde que isso traga ganhos claros de desempenho, escalabilidade horizontal e simplicidade no consumo da informação. Cada modelo NoSQL representa uma resposta diferente a esse desafio.

Modelagem Chave-Valor

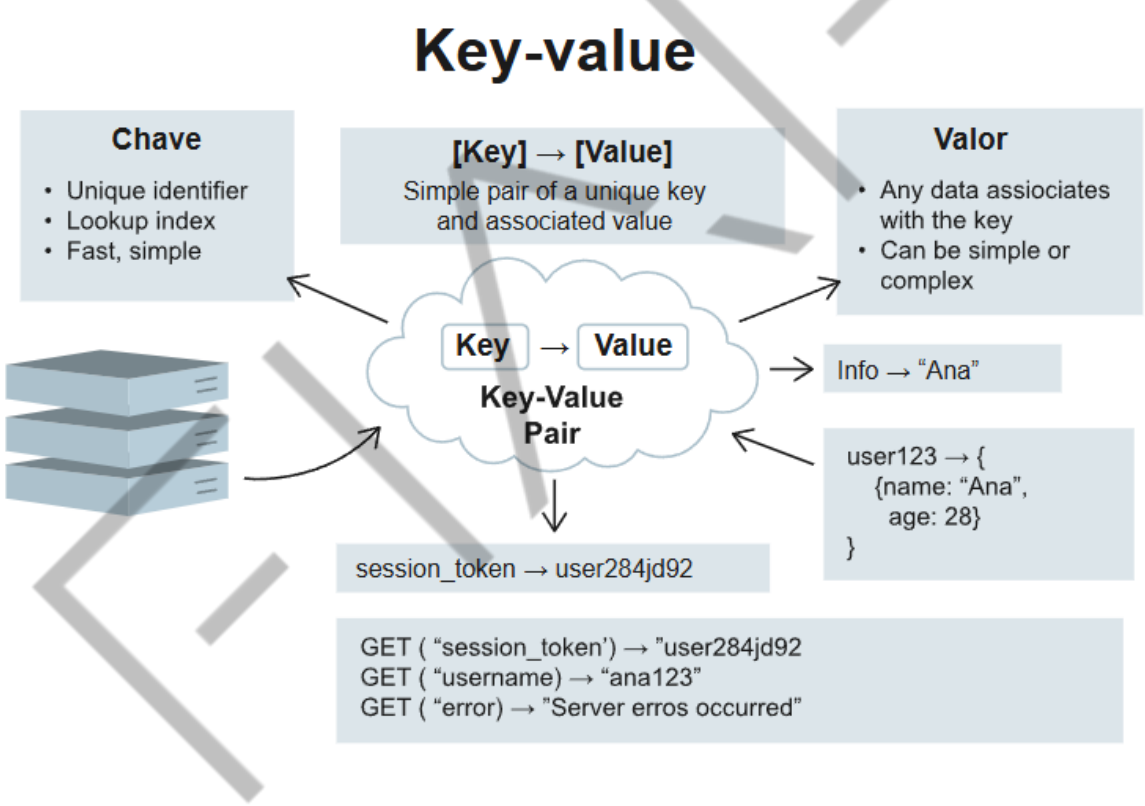


Figura 1 – Modelagem Chave-Valor

Fonte: elaborado pelo autor via Inteligência Artificial (2026)

O modelo **chave-valor** é a forma mais simples de armazenamento NoSQL. Cada dado é representado por uma chave única associada a um valor. Esse valor pode ser um dado simples (string, número) ou uma estrutura mais complexa, como um objeto ou um blob binário.

A grande força desse modelo está na **velocidade de acesso**. Como a chave funciona como um identificador direto, a busca é extremamente eficiente, tornando

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

esse tipo de banco ideal para cenários de cache, gerenciamento de sessão, controle de estado e leitura intensiva.

Do ponto de vista de modelagem, não existe estrutura interna imposta pelo banco. Isso significa que toda a semântica do dado fica sob responsabilidade da aplicação. Em ambientes analíticos, como no Databricks, esse modelo aparece quando trabalhamos com mapas (`MapType`) ou estruturas dinâmicas que não exigem schema fixo.

Um exemplo típico é o armazenamento de preferências de usuário, onde cada chave representa um usuário e o valor contém suas configurações. Essa abordagem evita joins, reduz latência e simplifica a escalabilidade horizontal. Em contrapartida, consultas complexas ou filtros sobre o conteúdo interno do valor não são o ponto forte desse modelo.

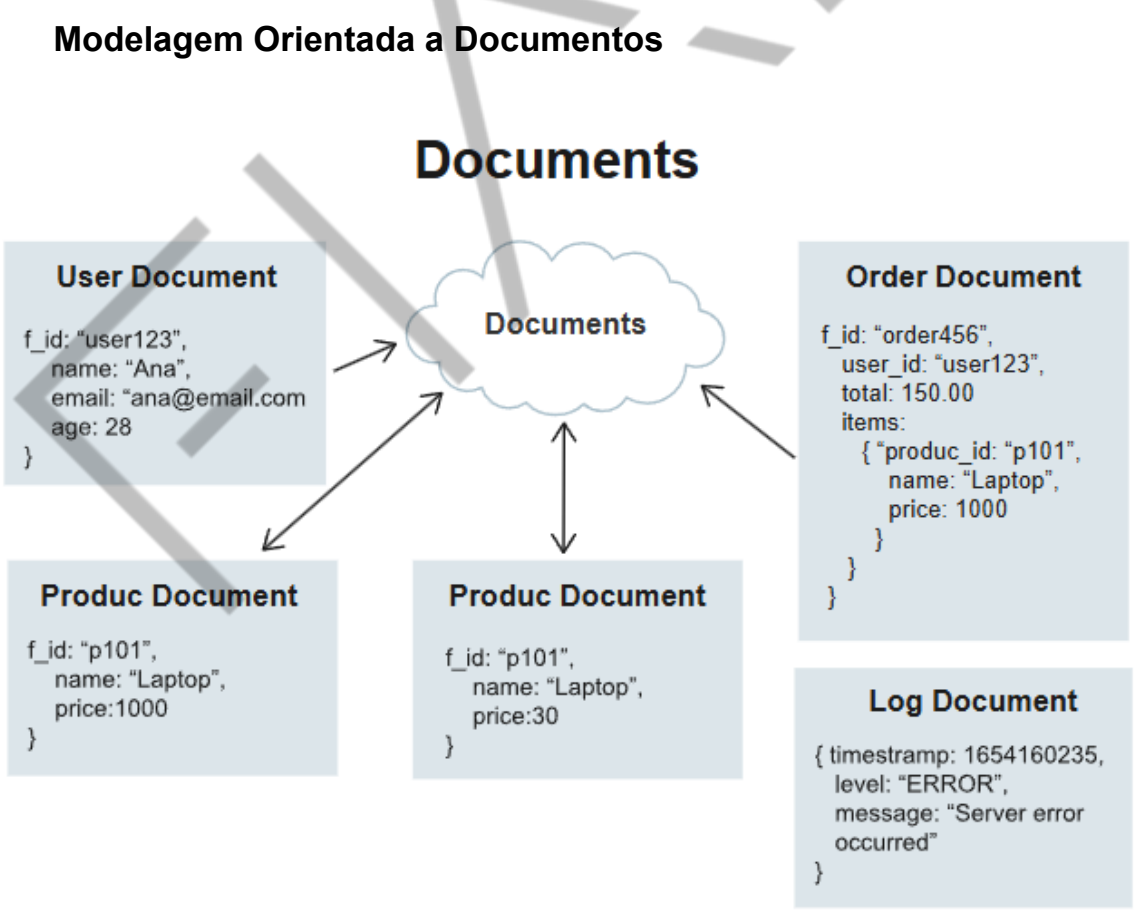


Figura 2 – Modelagem orientada a documentos

Fonte: elaborado pelo autor via Inteligência Artificial (2026)

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

No modelo **orientado a documentos**, os dados são armazenados como documentos autônomos, geralmente em formato JSON ou BSON. Cada documento representa uma entidade completa, contendo não apenas atributos simples, mas também estruturas aninhadas, listas e subdocumentos.

Esse modelo é especialmente poderoso quando os dados possuem **estrutura semiflexível** ou quando diferentes registros da mesma coleção não compartilham exatamente o mesmo conjunto de atributos. A modelagem se aproxima muito da forma como os dados são consumidos pela aplicação, reduzindo a necessidade de transformações intermediárias.

Um ponto central da modelagem em documentos é a decisão entre **embutir ou referenciar dados**. Por exemplo, ao modelar um cliente e seus pedidos, é possível armazenar todos os pedidos dentro do documento do cliente ou manter coleções separadas com referências. A escolha depende do volume de dados, da frequência de acesso e da necessidade de atualização independente.

Em plataformas analíticas, documentos aparecem naturalmente ao trabalhar com dados JSON, eventos de streaming, logs e payloads de APIs. O Databricks permite explorar esse modelo de forma eficiente, utilizando estruturas aninhadas e consultas sobre campos internos sem necessidade de normalização prévia.

Modelagem Orientada a Colunas

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

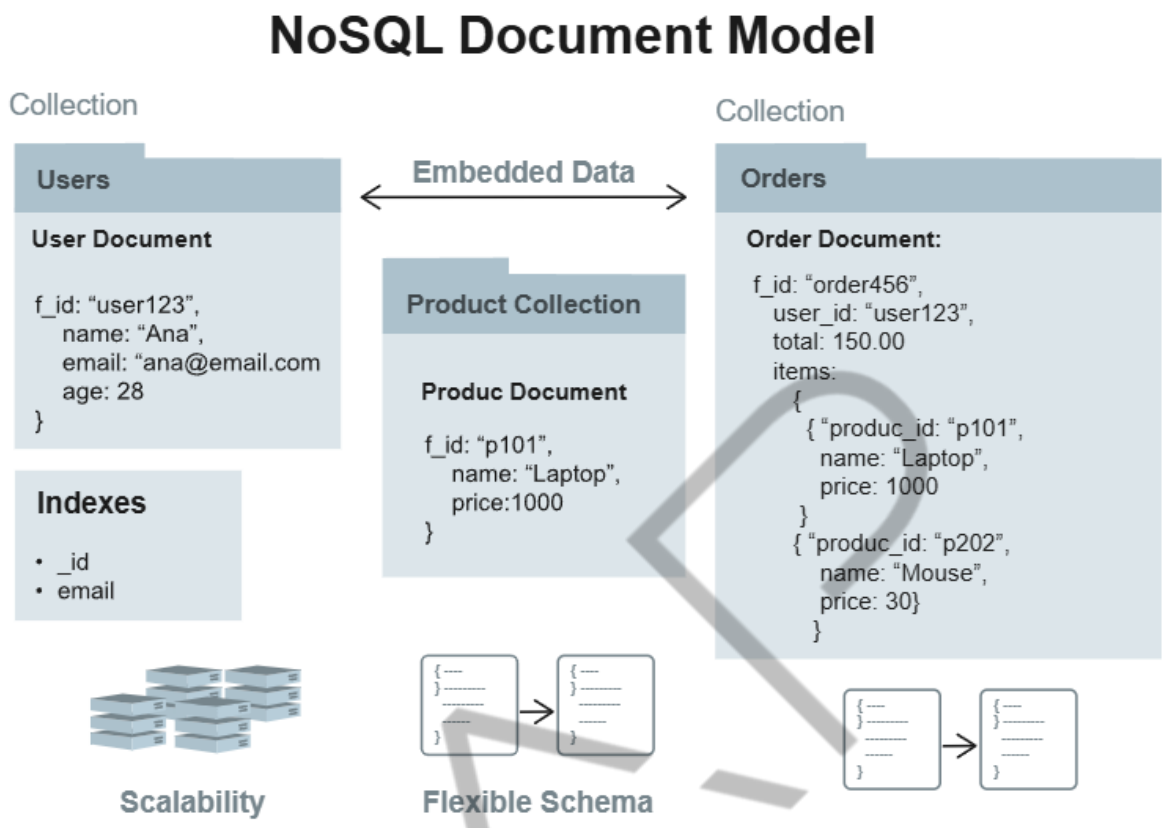


Figura 3 – Modelagem orientada a colunas

Fonte: elaborado pelo autor via Inteligência Artificial (2026)

O modelo **orientado a colunas**, também conhecido como *wide-column*, organiza os dados em famílias de colunas. Diferentemente do modelo relacional, onde todas as linhas compartilham o mesmo esquema, aqui cada linha pode conter um conjunto diferente de colunas dentro da mesma família.

Essa abordagem é extremamente eficiente para **grandes volumes de dados distribuídos**, especialmente quando as consultas acessam apenas subconjuntos específicos de colunas. Por isso, esse modelo é amplamente utilizado em sistemas de métricas, séries temporais, telemetria e análises históricas.

A modelagem em colunas exige uma mudança importante de mentalidade: o esquema é definido a partir das **consultas esperadas**, e não da entidade em si. Isso significa que tabelas são desenhadas para responder rapidamente a perguntas específicas, mesmo que isso implique duplicação de dados em diferentes estruturas.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

No Databricks, essa lógica se conecta diretamente ao armazenamento em formatos colunares como Parquet e Delta Lake, onde leituras seletivas de colunas reduzem I/O e melhoram significativamente o desempenho analítico.

Modelagem Orientada a Grafos

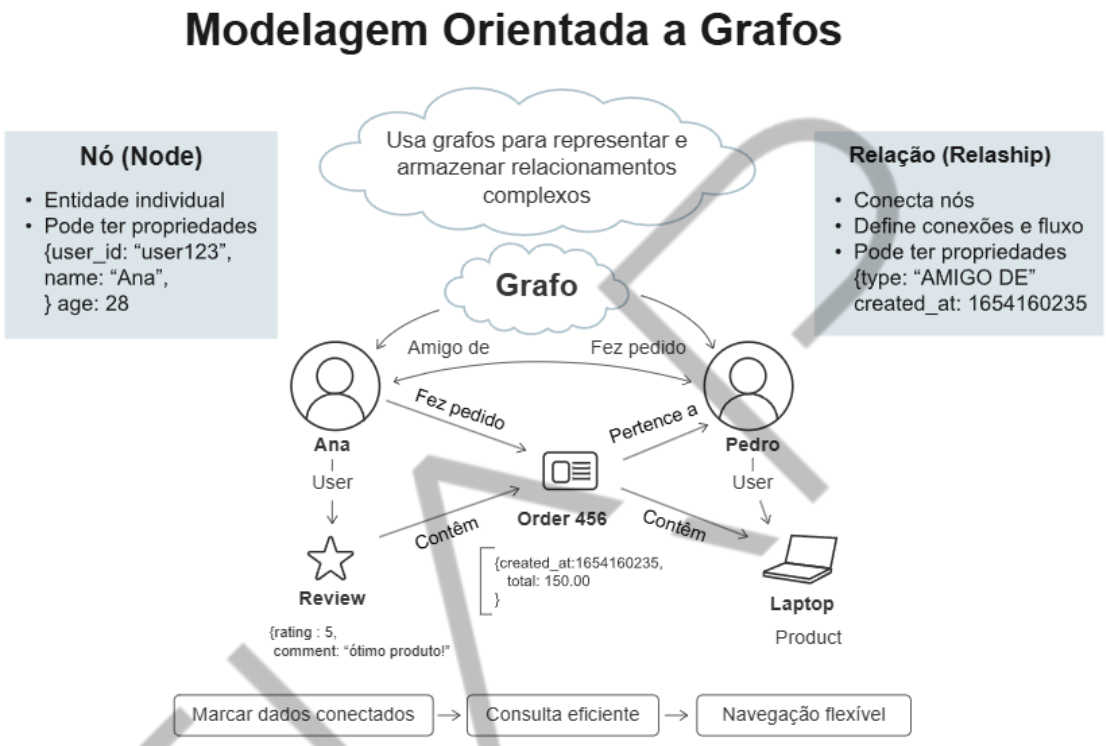


Figura 4 – Modelagem orientada a grafos

Fonte: elaborado pelo autor via Inteligência Artificial (2026)

O modelo **orientado a grafos** é voltado para cenários onde o principal valor está nas **relações entre entidades**, e não apenas nos dados isolados. Nesse modelo, os dados são representados por vértices (nós) e arestas (relacionamentos), ambos podendo conter propriedades.

A grande vantagem da modelagem em grafos é a eficiência na navegação por relacionamentos complexos. Consultas que exigiriam múltiplos joins em bancos relacionais tornam-se travessias naturais no grafo, com desempenho previsível mesmo em estruturas altamente conectadas.

Esse modelo é amplamente utilizado em redes sociais, sistemas de recomendação, detecção de fraude, análise de dependências e grafos de

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

conhecimento. A modelagem começa pela identificação das entidades centrais e dos tipos de relacionamento relevantes para o negócio.

Mesmo em ambientes analíticos tradicionais, conceitos de grafos podem ser simulados a partir de tabelas de vértices e arestas, como visto no hands on, permitindo análises avançadas de conectividade, influência e comunidades.

Comparando os modelos de NoSQL

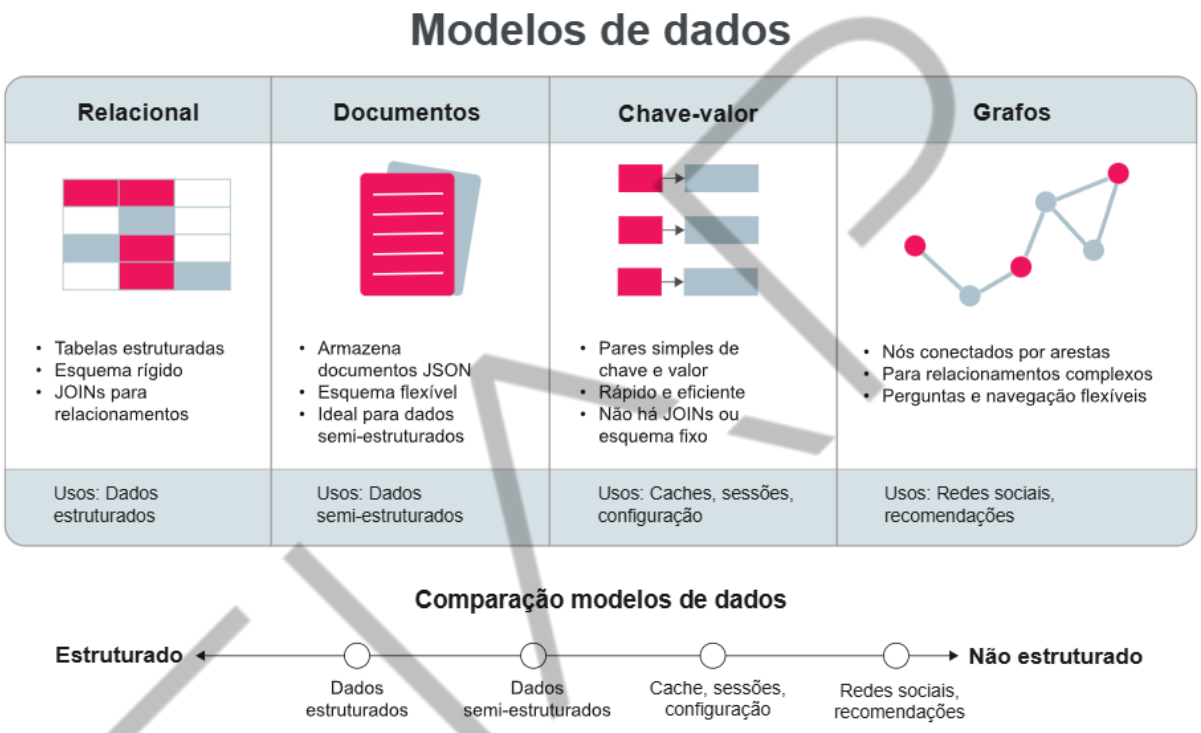


Figura 5 – Modelos de dados

Fonte: elaborado pelo autor via Inteligência Artificial (2026)

Cada modelo NoSQL resolve um tipo específico de problema. Bancos chave-valor priorizam velocidade extrema, bancos de documentos oferecem flexibilidade estrutural, bancos orientados a colunas escalam análises massivas e bancos de grafos dominam cenários relacionais complexos.

Não existe um modelo “melhor” de forma absoluta. Em arquiteturas modernas, é comum o uso combinado de múltiplos modelos, cada um atendendo a uma necessidade específica dentro do mesmo ecossistema. Essa abordagem é conhecida como **persistência poliglota**.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

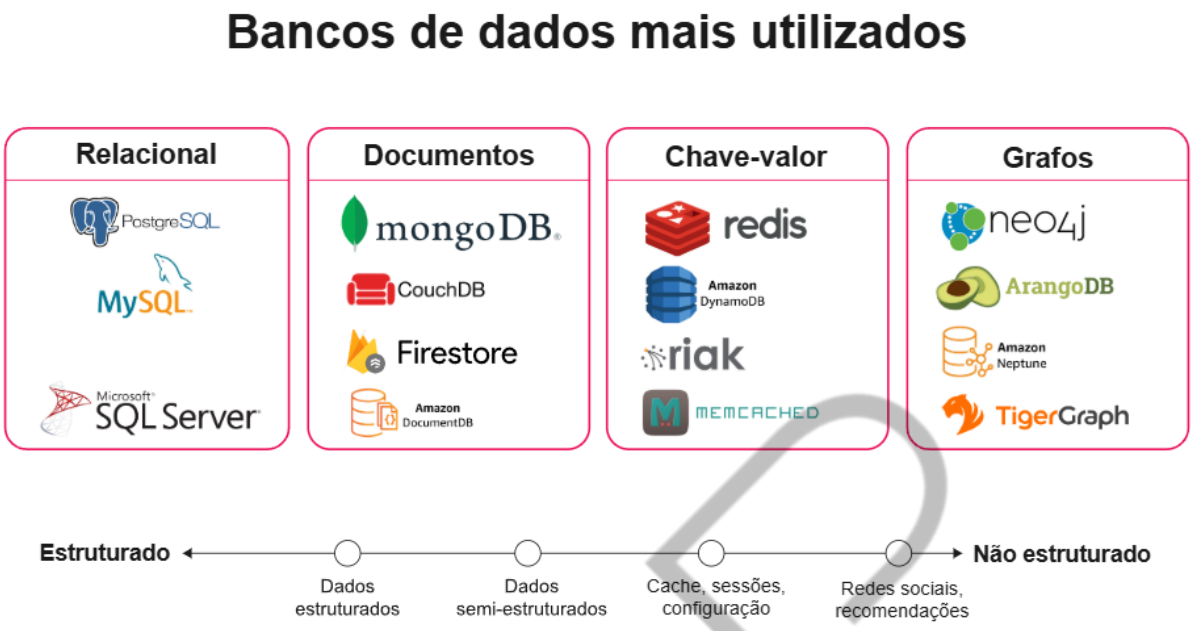


Figura 6 – Bancos de dados mais utilizados

Fonte: elaborado pelo autor via Inteligência Artificial (2026)

NoSQL e arquiteturas modernas de dados

A modelagem em NoSQL está profundamente conectada a arquiteturas como microsserviços, event-driven architectures e plataformas de analytics em nuvem. Nesses contextos, o banco de dados deixa de ser apenas um repositório e passa a ser um **componente ativo da arquitetura**, influenciando diretamente o design das aplicações.

Plataformas como o Databricks permitem que esses modelos coexistam no mesmo ambiente analítico, integrando dados estruturados, semiestruturados e relacionais em pipelines unificados. Isso amplia a capacidade de experimentação e acelera o ciclo entre modelagem, ingestão, análise e geração de valor.

Consolidando o aprendizado

Ao finalizar esta seção, retome os exemplos práticos e reflita sobre como cada decisão de modelagem impacta consultas, desempenho e manutenção. Experimente remodelar os mesmos dados sob diferentes abordagens NoSQL e observe como o foco deixa de ser a forma “correta” de modelar e passa a ser a forma **mais eficiente para resolver o problema proposto**.

Revise este conteúdo sempre que necessário e utilize-o como referência para suas próprias decisões de arquitetura e modelagem de dados.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

MERCADO, CASES E TENDÊNCIAS

A adoção de bancos de dados NoSQL consolidou-se como uma prática estratégica em empresas que operam com grandes volumes de dados, aplicações distribuídas e necessidade de resposta em tempo real. O mercado demonstra que o uso de NoSQL não está restrito a startups ou empresas de tecnologia, mas também a organizações tradicionais que buscam escalabilidade, resiliência e flexibilidade na modelagem de dados.

Empresas como a **Netflix** utilizam diferentes modelos NoSQL para lidar com catálogos de conteúdo, perfis de usuários e métricas de consumo em escala global. O uso combinado de bancos chave-valor e orientados a colunas permite reduzir latência, distribuir carga e garantir alta disponibilidade, mesmo em picos de acesso. Esse cenário evidencia a adoção da chamada **persistência poliglota**, em que diferentes bancos coexistem conforme o tipo de dado e padrão de acesso.

Um panorama claro sobre como bancos NoSQL são aplicados no mundo real pode ser encontrado no artigo *NoSQL Databases: Real-World Use Cases of MongoDB, Cassandra and Redis*, que apresenta exemplos práticos de uso desses bancos em aplicações modernas, destacando diferenças de modelagem e cenários ideais de adoção. Disponível em: https://medium.com/@priyanshu011109/nosql-databases-demystified-real-world-use-cases-of-mongodb-cassandra-and-redis-669e2d33ac89.

No contexto de bancos orientados a documentos, o **MongoDB** é amplamente adotado por empresas de e-commerce, mídia digital e plataformas SaaS. A flexibilidade do modelo de documentos permite que catálogos de produtos, perfis de clientes e registros de eventos sejam modelados de forma aderente ao consumo das aplicações, reduzindo a necessidade de transformações complexas.

O artigo *10 Real-World MongoDB Use Cases in 2024* apresenta exemplos concretos de empresas que utilizam MongoDB para lidar com dados semiestruturados, personalização e alta escalabilidade. O conteúdo reforça como decisões de modelagem impactam diretamente desempenho e evolução do sistema. Disponível em: https://www.cdata.com/blog/mongodb-use-cases.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

No modelo chave-valor, o **Amazon DynamoDB** é um dos principais exemplos de adoção em larga escala. Empresas utilizam o DynamoDB para aplicações críticas que exigem escalabilidade automática, como carrinhos de compra, gerenciamento de sessões e processamento de eventos em tempo real. A modelagem, nesse contexto, é fortemente orientada à chave de acesso e aos padrões de leitura.

O artigo *Top Use Cases for DynamoDB* apresenta cenários reais de uso do DynamoDB em sistemas distribuídos, explorando como a escolha da chave de partição influencia diretamente a performance e a escalabilidade. Disponível em: https://www.tinybird.co/blog/dynamodb-use-cases.

Bancos orientados a grafos também vêm ganhando espaço no mercado, especialmente em aplicações que exigem análise de relacionamentos complexos. O **Neo4j** é amplamente utilizado em sistemas de recomendação, detecção de fraude e análise de redes. Empresas de tecnologia e instituições financeiras adotam grafos para identificar padrões que seriam difíceis ou custosos de modelar em bancos relacionais.

O artigo *Graph Databases, NoSQL, and Neo4j* explica como bancos de grafos se inserem no ecossistema NoSQL e apresenta exemplos de aplicações reais, reforçando o papel da modelagem orientada a relacionamentos. Disponível em: https://neo4j.com/news/graph-databases-nosql-and-neo4j/.

Do ponto de vista de arquitetura e tendências, observa-se uma forte integração entre bancos NoSQL e plataformas analíticas modernas. Dados operacionais oriundos de NoSQL são cada vez mais consumidos em arquiteturas analíticas para machine learning, inteligência artificial e análises avançadas. Esse movimento reforça a importância de uma modelagem bem planejada desde a origem do dado.

Uma visão consolidada sobre os principais casos de uso de NoSQL no mercado pode ser encontrada no material *NoSQL Database Use Cases*, publicado pela ScyllaDB, que apresenta cenários reais em áreas como IoT, sistemas financeiros e aplicações em tempo real. Disponível em: https://www.scylladb.com/learn/nosql/nosql-database-use-cases/.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Podcasts e livros recomendados

Como complemento ao conteúdo técnico, o podcast *Graph Databases Explained*, produzido pela equipe do Neo4j, discute casos reais e decisões de modelagem em bancos orientados a grafos, conectando teoria e prática de mercado. Disponível em: https://neo4j.com/podcast/.

Entre as leituras fundamentais, destaca-se o livro **Designing Data-Intensive Applications**, de Martin Kleppmann (O'Reilly, 2017), que aborda profundamente os princípios de sistemas distribuídos, NoSQL, escalabilidade e consistência. Outra obra relevante é **NoSQL Distilled**, de Pramod J. Sadalage e Martin Fowler (Addison-Wesley, 2013), que apresenta uma visão clara e objetiva sobre os principais modelos NoSQL e seus contextos de uso.

Ao analisar o mercado, fica evidente que **modelagem de dados em NoSQL é uma competência estratégica**, diretamente ligada à capacidade das empresas de escalar, inovar e responder rapidamente às demandas do negócio. Compreender como essas organizações modelam seus dados permite ao profissional de dados tomar decisões mais assertivas em projetos reais.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você compreendeu como a **modelagem de dados em NoSQL** se diferencia do modelo relacional tradicional, adotando uma abordagem orientada ao problema, ao padrão de acesso e à escalabilidade. Foram apresentados os principais modelos NoSQL — **chave-valor, documentos, colunas e grafos** — destacando como cada um organiza os dados e em quais cenários são mais adequados.

Ao longo do conteúdo, você analisou **exemplos práticos de modelagem**, desde estruturas simples de chave-valor até representações mais complexas baseadas em documentos, famílias de colunas e grafos de relacionamento. Também foi possível entender como essas modelagens são aplicadas em plataformas modernas, como ambientes analíticos e arquiteturas distribuídas, conectando teoria e prática.

Além disso, apresentamos **casos reais do mercado**, mostrando como empresas utilizam diferentes bancos NoSQL para atender demandas de alta escala, baixa latência e análise de dados em tempo real.

Ao final, você consolidou a visão de que não existe um modelo único ou “melhor”, mas sim a necessidade de **escolher conscientemente a estratégia de modelagem** que melhor se adapta ao contexto do negócio e às exigências das aplicações modernas.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

REFERÊNCIAS

CDATA SOFTWARE. **10 real-world MongoDB use cases in 2024**. 2024. Disponível em: https://www.cdata.com/blog/mongodb-use-cases. Acesso em: 26 mar. 2026.

KLEPPMANN, M. **Designing data-intensive applications**. Sebastopol: O'Reilly Media, 2017.

NEO4J. **Graph databases, NoSQL, and Neo4j**. 2023. Disponível em: https://neo4j.com/news/graph-databases-nosql-and-neo4j/. Acesso em: 26 mar. 2026.

PRINCE, P. **NoSQL databases demystified: real-world use cases of MongoDB, Cassandra and Redis**. Medium, 2023. Disponível em: https://medium.com/@priyanshu011109/nosql-databases-demystified-real-world-use-cases-of-mongodb-cassandra-and-redis-669e2d33ac89. Acesso em: 26 mar. 2026.

ROBINSON, I.; WEBBER, J.; EIFREM, E. **Graph databases**. 2. ed. Sebastopol: O'Reilly Media, 2015.

SADALAGE, P. J.; FOWLER, M. **NoSQL distilled: a brief guide to the emerging world of polyglot persistence**. Boston: Addison-Wesley, 2013.

SCYLLADB. **NoSQL database use cases**. 2024. Disponível em: https://www.scylladb.com/learn/nosql/nosql-database-use-cases/. Acesso em: 26 mar. 2026.

TINYBIRD. **Top use cases for DynamoDB**. 2024. Disponível em: https://www.tinybird.co/blog/dynamodb-use-cases. Acesso em: 26 mar. 2026.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

PALAVRAS-CHAVE

NoSQL, Modelagem De Dados, Persistência Poliglota.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com



PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com